The registry book: federation, aliases, credentials, the official seed

TBK SPEC 37:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 37:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Why (the two hosting worlds, one machine) 5

2. The registry book (the config model) 5

3. Name resolution (bare, qualified, pinned) 6

4. The federation grammar (spec 04 §1 amendment) 7

5. The credential model (locked confinement) 8

6. The official seed (spec 16 amendment) 9

7. Origin binding (upgrade confinement) 9

8. Private runtimes (schema MINOR 1, shipped by this spec) 10

9. Internal instances (the three operator tiers) 10

10. Named errors (new or re-scoped here) 11

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: SHIPPED — §2's registry book (the config model, alias grammar + derivation, `DuplicateRegistryAlias` / `DuplicateDefaultRegistry`, `add-registry --name/--require-signed/ --default`, the `list-registries` book rendering), §2.2's per-registry fail-closed signature policy (`UnsignedRegistryPayload`, exit 70), §3's qualified form (`alias/name` + the `registry:` pin scopes), §4's federation grammar (the explicit-host `tfs+<svc>://host/...` forms, the `tfs+https://` registry location, the `UnsupportedService` / `SshTransportUnsupported` refusals), §5's credential model (the two-tier book, the locked confinement, the fetch journal, `CredentialRequired` / `InvalidCredentialEntry` / `DuplicateCredentialSelector`), §6's official seed (`tebako setup` and the installer's written config; the tebako-packages/registry aggregate flowing the feedstocks' authored rows), §7's origin binding (the `.tfs.registry` marker, the confined version chains, the journaled rebind), §8's private runtimes (the DEPENDS-edge `registry:` pin — payload-manifest schema_minor 14 — the federated release-download locator deriving `kind: runtime` rows on GitHub/GHE/GitLab, and `require_signed` riding runtime rows), and §9's three operator tiers (their resolution paths — `tfs+https://`, explicit-host git services, `file://` + bundles — all ride the shipped grammar; the tier how-tos live on tebako.org). (Locked direction 2026-09-24; design reviewed with the owner across the 2026-09-24 session — the alias grammar, the credential confinement model, and the federation forms below are the agreed shape). Nothing here changes what shipped resolvers do with a config that carries none of the new keys.

Normative specification of the registry book: the named, policy-bearing list of registries in `~/.tebako/config.yaml` that turns spec 04 §2's "any git host repo" into a federation — public official payloads and private runtime/executable/data payloads resolving side by side on one machine, with leak-proof credentials and confusion-proof names. The registry stays developer-hosted zero-infrastructure YAML; this spec adds the bookkeeping that makes MANY of them safe to mix.

© 2026 The Tebako Project

1. Why (the two hosting worlds, one machine)

A real deployment mixes two worlds: public payloads from the official registry (Homebrew's core-tap role) and private payloads from an operator's own instance (Docker's enterprise-registry role) — a consultancy's GitHub for public tools, their own GitLab for client work; Ribose's public tebako-packages org, plus gated flavor registries where BSI's tokens must never read NIST's repos. Today's model already fails-closed on ambiguity (spec 04 §2: N registries carrying one name is the named `AmbiguousRegistries`, never a priority pick) — but it has no aliases for ergonomic scoping, no per-registry policy, credentials only for github.com, and no grammar for a service API at an operator's OWN host. This spec closes exactly those four gaps. What it never adds: a resolution priority, a default service compiled into any binary, a silent fallback.

2. The registry book (the config model)

`registries:` entries gain OPTIONAL keys; a bare string entry keeps meaning what it means today (spec 04 §2, schema-compatible — old readers see the same refs):

FIGURE 1

```
# ~/.tebako/config.yaml
registries:
  - name: official                          # the local alias
(§3's handle)
    ref: tfs:github:tebako-packages/registry
    default: true                           # publish/UX anchor
ONLY — §2.1
  - name: nist
    ref: tfs:github:metanorma/metanorma-flavor-nist
    require_signed: true                    # fail-closed trust
policy — §2.2
```

- Alias grammar: `name` matches `[a-z][a-z0-9-]*`, unique across the book; when omitted, the alias DERIVES from the ref's `owner/repo` as `repo` (a collision with an existing alias is the named `DuplicateRegistryAlias` at config load, listing both entries — never a silent rename). Aliases are LOCAL names for local config — they are never published, never embedded in payloads, never part of any wire format.
- 2.1 The `default` flag is a publish/UX anchor, never a resolution tiebreaker. Exactly one entry MAY carry `default: true` (two → `DuplicateDefaultRegistry`, config-

© 2026 The Tebako Project

load time). The default registry is (a) the target of `tebako publish` when `--registry` is absent, (b) rendered first in listings and in the registry-listing body of named errors. Resolution has NO default: §3's rules are identical whether or not a default exists. (Invariant 9 stands: no default SERVICE; a default LABEL on user-owned config is not a service.)

- 2.2 `require_signed: true` makes every install whose plan resolves through that registry fail-closed unless the resolved row carries a `signature:` block that verifies against the machine's trust anchors (spec 09) — the registry-level form of `TEBAKO_REQUIRE_SIGNED=1`, scoped so a private operator can demand signatures on THEIR payloads without unsigned public payloads becoming second-class. A per-registry `keyring: <ref>` (PLANNED with the trust-anchor distribution work, spec 09 §10) pins WHICH keys count; absent it, the machine trust store decides.
- `tebako add-registry` grows `--name <alias>`, `--require-signed`, `--default`; `list-registries` renders the book (alias, ref, policy flags, cache freshness). The shipped config still has ZERO registries (§6's seed is the installer's authored content, not a compiled-in value).

3. Name resolution (bare, qualified, pinned)

Three forms, each with exactly one home — the ambiguity law is unchanged:

1. Bare `name` — searches EVERY registered registry (shipped behavior, locked): zero hits → the named not-found error listing the registered registries; exactly one → plan; more than one → `AmbiguousRegistries` naming every carrier. No priorities, no first-hit, in either direction — a public registry can never shadow a private name NOR the reverse.
2. Qualified `alias/name` (NEW) — scopes resolution to the one registry whose alias is `alias`. Unknown alias → the named `UnknownRegistryAlias` listing the book's aliases. This is the ergonomic disambiguator short of a full reference (the Cargo `registry = "..."` / Homebrew `tap/formula` role): `tebako install nist/metanorma-bsi` can never touch the official registry's rows.
3. Full reference — unchanged (spec 04 §1); no registry involved at all.

Pin surfaces (`.tebako-tools.yaml`, the config's per-tool `version:` pins, spec 23 composition edges) gain an optional `registry: <alias>` key naming the registry the pin resolves through — the same scoping, authored. A pin naming an alias not in the book is `UnknownRegistryAlias` at pin-resolution time, fail-closed.

© 2026 The Tebako Project

4. The federation grammar (spec 04 §1 amendment)

The service adapters' canonical SaaS hosts stay the bare forms; the operator's own host is EXPLICIT in the scheme — the adapter is still never inferred:

FIGURE 2

```
tfs:github:owner/repo:ver[#asset]              → github.com
(unchanged)
tfs+github://ghe.corp.internal/owner/repo:ver[#asset]
    → the GitHub API shape at that host (GitHub Enterprise
Server;
      base https://<host>/api/v3). ALSO the Gitea/Forgejo
form — those
      services implement the GitHub releases/contents API; a
CI
      conformance leg proves the claim, the grammar never
special-cases
      them.
tfs+gitlab://gitlab.corp.internal/group/sub/repo:ver[#asset]
    → GitLab API v4 at that host (self-hosted; nested groups
unchanged)
tfs+https://artifacts.corp.internal/tebako/tpkg-registry.yaml
    → NEW REGISTRY LOCATION: the registry file itself over
plain HTTPS
      (spec 04 §2's "no other locations" is amended by THIS
form only —
      a static server, an S3 bucket behind a CDN, an
Artifactory/Nexus
      generic repo). Artifact refs inside such a registry
are ordinary
      spec 04 references — typically sibling `tfs+https:`
URLs.
```

- Bitbucket Data Center is NOT a host variant of `tfs:bb:` — its API differs from Bitbucket Cloud's. It is the named `UnsupportedService` ("bitbucket data center is not bitbucket cloud; host the registry over tfs+git: or tfs+https:") until a real consumer funds the adapter.
- SSH is the named `SshTransportUnsupported` ("git over ssh needs the system ssh binary — forbidden by the no-shell-outs law; use a token over https, or file:"). A pure-Rust SSH transport (russh-class) can land later behind the SAME `tfs+git:` grammar with no reference change; until then, fail closed by name.
- Host-parameterized adapters are ONE code path with a base-URL parameter — the SaaS forms are the parameter's canonical value, never a second implementation to drift.

© 2026 The Tebako Project

5. The credential model (locked confinement)

FIGURE 3

```
credentials:
  - registry: nist                # tier 1: exact registry-
alias match
    token_env: NIST_GH_TOKEN
  - host: ghe.corp.internal       # tier 2: host fallback
    token_env: GHE_PAT
```

- **Two tiers, most-specific-wins, exactly one match.** A fetch looks up tier 1 by the alias of the registry whose row directed the fetch (registry-file fetches match their own alias), then tier 2 by the URL's host, then anonymous. There is no third tier and no chain.
- **The registry is the trust unit, the host is the convenience.** The decisive case: two flavor registries on github.com whose tokens must not cross-read — per-host keying cannot express it, per-alias can. Two registries sharing one token name the same `token_env` twice; repetition is the price of a match rule with zero precedence ambiguity.
- **Config holds env var NAMES, never secrets.** A credential whose env var is unset behaves as absent (the fetch goes anonymous → the service's ordinary 401/403 → `CredentialRequired` naming the registry and the env var it looked for).
- **Confinement (locked, generalizing the 2026-09-15 GitHub amendment to every adapter):** a credential is presented ONLY to the host it was minted for — tier 1 credentials to the registry ref's own host (plus that service's API host, per the service's documented mapping), tier 2 to the named host — NEVER to a redirect target, NEVER cross-host. A registry row whose `release.ref` points at a DIFFERENT host falls to that host's own tier-2 entry or fails named; the directing registry's credential does not follow.
- **The adapter owns the header shape** by service class (GitHub/GHE: `Authorization: Bearer`; GitLab: `PRIVATE-TOKEN`; Bitbucket Cloud: basic). Authored config never spells header mechanics.
- Every fetch journals host + credential CLASS (`token:env:NIST_GH_TOKEN`, `anonymous`) with the value REDACTED — the mixed-federation audit trail. The ambient `TEBAKO_GITHUB_TOKEN/GITHUB_TOKEN` amendment is unchanged and ranks as tier 2 for github.com when no tier-1 entry matched.

© 2026 The Tebako Project

6. The official seed (spec 16 amendment)

The installer's written config (and `tebako setup` on a config-less machine) SEEDS the official entry — authored, user-visible, user-removable config content:

FIGURE 4

```
registries:
  - name: official
    ref: tfs:github:tebako-packages/registry
    default: true
```

- No binary carries this value; `tebako` built from source with no installer still ships zero registries. The seed is DISTRIBUTION content (spec 16's install.sh / setup flow), exactly how Homebrew's installer clones the core tap without brew hardcoding it.
- The official registry repo is a GENERATED AGGREGATE: each feedstock repo's own `tpkg-registry.yaml` remains the authored authority (SSOT); the aggregate is CI-flowed from them with parity asserted per row — never hand-edited. Removal/withdrawal flows the same way (`status: withdrawn` rows aggregate verbatim).
- `tebako bundle --config <org.yaml>` (spec 16 §4) overlays registries as today — the airgapped/enterprise image-builder path is untouched; its overlay can REPLACE the seeded book wholesale.

7. Origin binding (upgrade confinement)

A store entry's markers already record WHERE bytes came from; the book adds WHICH REGISTRY resolved them — the `.tfs.registry` marker beside `.tfs.origin`, carrying the resolving registry's canonical REFERENCE (never an alias; aliases are local and volatile). Dispatch-time version chains (spec 07 §1's chain, spec 05 §5) and unscoped install-time name searches alike consult the payload's ORIGIN registry only: a same-named payload appearing in another registered registry is never a silent "upgrade" of an installed one (the confusion attack that priority-based ecosystems — rubygems sources, docker's implicit docker.io — leak by construction). An authored `registry:` scope (§3) outranks the binding. Switching registries is an explicit act: `tebako install <alias>/<name>` reinstalls and rebinds the origin; the journal records the rebind (`event=origin-rebind`).

© 2026 The Tebako Project

A rebind whose registry-declared digest disagrees with the cached bytes is refused (`Sha256Mismatch`, exit 70) — the cache never overwrites a standing entry, so bytes are never re-tagged to a registry that never published them: uninstall first, then install from the new registry.

8. Private runtimes (schema MINOR 1, shipped by this spec)

Spec 04 §2's `kind: runtime` registry entries move PLANNED → SHIPPED: a registry row of kind runtime carries `engine:` (+ optional `implementation:`), and a payload's kind-runtime DEPENDS edge (spec 30 §1) discovers it by (engine, implementation?, constraint) across the REGISTERED book under §3's rules (qualified form included: `runtime_requirement` gains no alias field — the EDGE carries `registry:` per §3's pin-surface rule). The matched version's `release.ref` derives the download base exactly as spec 05 §2's per-engine chain; the trust pipeline (sha pins, bundle-era rows, `require_signed`) is identical to the factory index's. A private ruby or openjdk line is thereby hostable next to public payloads with zero product-side special cases.

9. Internal instances (the three operator tiers)

The grammar above makes three hosting tiers first-class; the HOW-TO lives on tebako.org, the normative part is only that these resolve:

1. Zero software — a static HTTPS server (nginx, S3+CDN, Artifactory/Nexus generic repo) serving `*.tfs` + sidecars + one `tpkg-registry.yaml`, registered as `tfs+https://…/tpkg-registry.yaml`.
2. The git host they already run — GHE / self-hosted GitLab / Gitea / Forgejo via §4's explicit-host forms; releases are the artifact storage exactly as on the SaaS hosts.
3. Airgap — `file://` mirror or a `tebako bundle` pre-seeded store (shipped; spec 16 §4), no network surface at all.

All three compose with the enterprise-networking amendment (spec 04 §4: proxies, custom CA roots) unchanged — internal TLS inspection is a transport concern the federation grammar never sees.

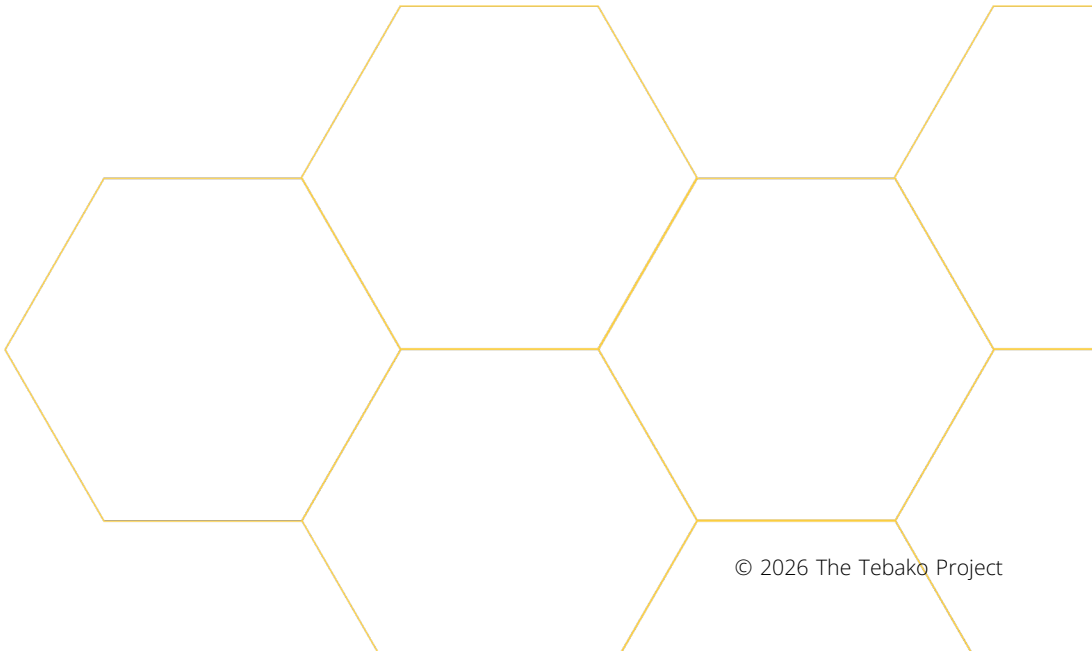
© 2026 The Tebako Project

10. Named errors (new or re-scoped here)

TABLE 1

Name	Exit class	Moment
`UnknownRegistryAlias`	65 usage	`alias/name` or `registry:` pin names no book entry — lists aliases
`DuplicateRegistryAlias`	65	config load; both entries named
`DuplicateDefaultRegistry`	65	config load; two `default: true`
`AmbiguousRegistries`	65	(shipped) bare name in >1 registry — the anti-confusion anchor
`UnsupportedService`	65	Bitbucket Data Center refs — the tfs+git:/tfs+https: steer
`SshTransportUnsupported`	65	any ssh/git@ URL — the token-over-https steer
`CredentialRequired`	69 unavailable	401/403 with no matching credential — names the registry + env var looked for
`InvalidCredentialEntry`	65 usage	config load; a credentials entry names both `registry:` and `host:`, neither, or a malformed `token_env`
`DuplicateCredential Selector`	65	config load; two entries key the same alias or host
`UnsignedRegistryPayload`	70 integrity	`require_signed` row without a verifying signature — fail-closed
`Sha256Mismatch`	70 integrity	(re-scoped from spec 04 §3) a rebind whose registry-declared digest disagrees with the cached bytes — uninstall first

Each maps to exactly one CI tier's assertion; none is a fallback — a failure here changes NOTHING about what is installed or resolved elsewhere.



© 2026 The Tebako Project